



IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MEMÓRIA COMPARTILHADA

OPENMP

Prof. Samuel Xavier de Souza
Prof. Carla Santana

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- Objetivos deste tópico
 - Compreender o paradigma de memória compartilhada
 - Compreender os desafios de programar em memória compartilhada
 - Conhecer as ferramentas básicas de programação em memória compartilhada
 - `#pragma omp critical`
 - `#pragma omp parallel`
 - `#pragma omp for`
 - cláusulas `private`, `firstprivate`, `lastprivate` e `shared`.
 - cláusula `default(none)`

Número de threads na região paralela

- Variável de ambiente **OMP_NUM_THREADS**
 - **export OMP_NUM_THREADS =n**
- Função **omp_set_num_threads(n)**
- Cláusula **num_threads(n)**

Diretiva parallel

```
#pragma omp parallel [clauses]
{
    bloco parelo
}
```

```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) { /* omp_parallel .c */
    # pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        printf (" Alo da thread %d. \n", tid );
    }
    return 0;
}
// Compilar
// gcc -o my_omp_parallel_program omp_parallel .c - fopenmp
// Executar
// ./my_omp_parallel_program
```

Diretiva parallel

```
#pragma omp parallel [clauses]
{
    bloco paralelo
}
```

```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) { /* omp_parallel .c */
    # pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        printf (" Alo da thread %d. \n", tid );
    }
    return 0;
}
// Compilar
// gcc -o my_omp_parallel_program omp_parallel .c - fopenmp
// Executar
// ./my_omp_parallel_program
```

Execute o código ao lado e, em seguida, acrescente um laço **for** dentro da região paralela. Depois, imprima qual índice cada thread está processando.

Diretiva parallel

```
#pragma omp parallel [clauses]
{
    bloco paralelo
}
```

```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) { /* omp_parallel .c */
    # pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        printf ( " Alo da thread %d. \n", tid );
        for (int i = 0; i < 12; i++)
        {
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );
        }
    }
    return 0;
}
```

Nesse código todas as threads executarão o loop todo

Diretiva for



```
#pragma omp for [clauses]  
for
```



```
#include <stdio.h>  
#include <omp.h>  
int main ( int argc , char * argv []) { /* omp_parallel .c */  
    # pragma omp parallel num_threads (4)  
    {  
        int tid = omp_get_thread_num () ;  
        printf ( " Alo da thread %d. \n", tid );  
        #pragma omp for  
        for (int i = 0; i < 12; i++)  
        {  
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );  
        }  
    }  
    return 0;  
}
```

Diretiva for



```
#pragma omp for [clauses]  
for
```



```
#include <stdio.h>  
#include <omp.h>  
int main ( int argc , char * argv []) { /* omp_parallel .c */  
    # pragma omp parallel num_threads (4)  
    {  
        int tid = omp_get_thread_num () ;  
        printf ( " Alo da thread %d. \n", tid );  
        #pragma omp for  
        for (int i = 0; i < 12; i++)  
        {  
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );  
        }  
    }  
    return 0;  
}
```

Execute o código ao lado
e, em seguida, retire a
diretiva parallel deixando
somente a **for**.

Diretiva for

```
#pragma omp parallel for [clauses]  
for
```

```
#include <stdio.h>  
#include <omp.h>  
int main ( int argc , char * argv []) { /* omp_parallel .c */  
    # pragma omp parallel num_threads (4)  
    {  
        int tid = omp_get_thread_num ( ) ;  
        printf ( " Alo da thread %d. \n", tid );  
        #pragma omp for  
        for (int i = 0; i < 12; i++)  
        {  
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );  
        }  
    }  
    return 0;  
}
```

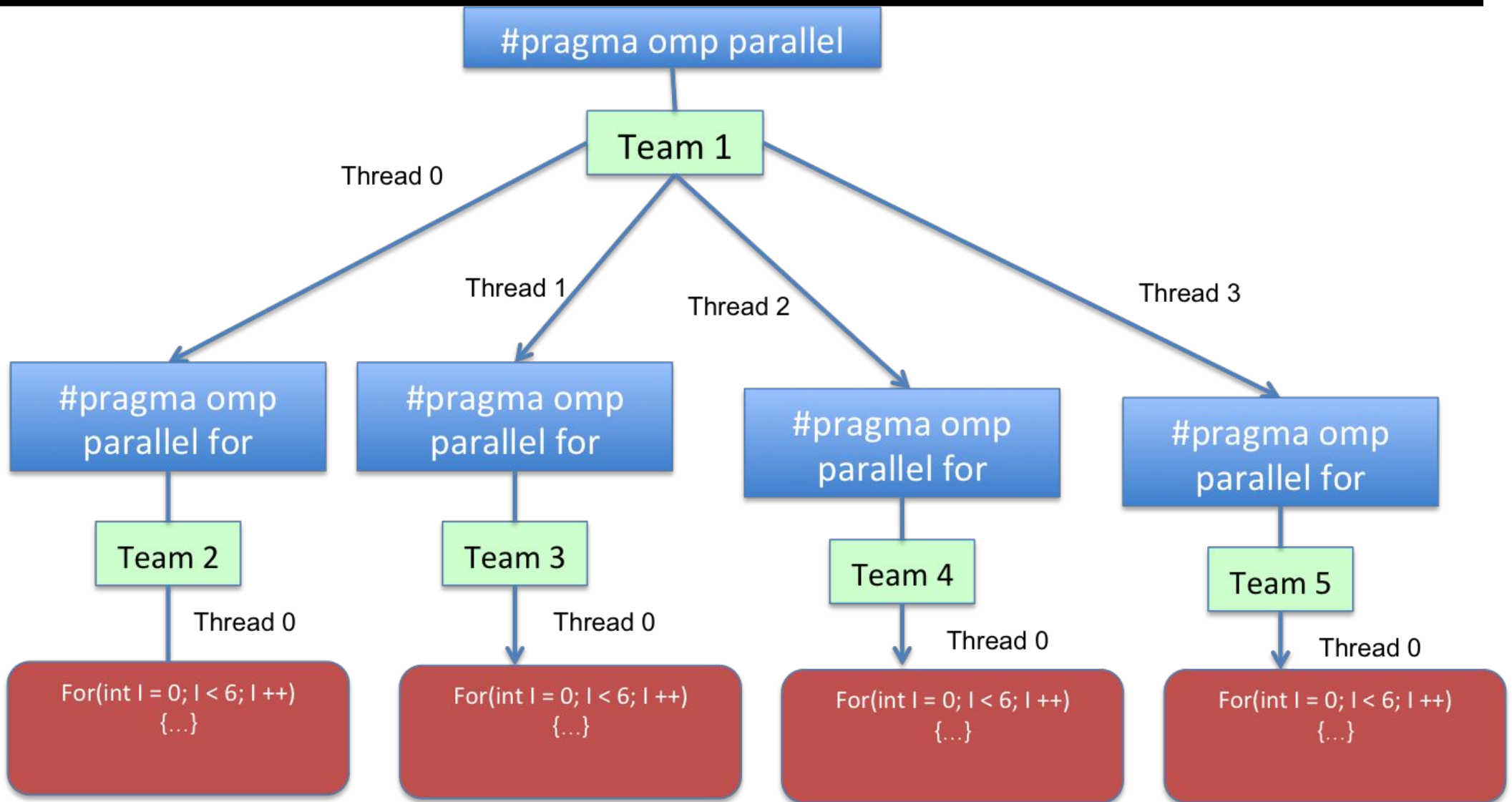
Execute o código ao lado
e, em seguida, retire a
diretiva parallel deixando
somente a **for**.

Diretiva for

```
#pragma omp parallel for [clauses]  
for
```

```
#include <stdio.h>  
#include <omp.h>  
int main ( int argc , char * argv []) { /* omp_parallel .c */  
    # pragma omp parallel num_threads (4)  
    {  
        int tid = omp_get_thread_num ( ) ;  
        printf ( " Alo da thread %d. \n", tid );  
        #pragma omp for  
        for (int i = 0; i < 12; i++)  
        {  
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );  
        }  
    }  
    return 0;  
}
```

*E se dentro de uma
região paralela a gente
criar outra região
paralela?*



Diretiva for

```
#pragma omp parallel for [clauses]  
for
```

```
#include <stdio.h>  
#include <omp.h>  
int main ( int argc , char * argv []) { /* omp_parallel .c */  
    # pragma omp parallel num_threads (4)  
    {  
        int tid = omp_get_thread_num ( ) ;  
        printf ( " Alo da thread %d. \n", tid );  
        #pragma omp for  
        for (int i = 0; i < 12; i++)  
        {  
            printf ( " Alo da thread %d. fazendo o trabalho %d \n", tid, i );  
        }  
    }  
    return 0;  
}
```

Pesquise sobre
omp_nested .

Cláusula `shared`, `private`, `default`, `firstprivate`

- **`shared`**: a variável é acessada por todas as threads
- **`private`**: cada thread possui sua própria cópia da variável, não visível para as demais
- **`default (shared|none)`**
 - **`shared`**: as variáveis não especificadas explicitamente serão compartilhadas por padrão.
 - **`none`**: todas as variáveis devem ter seu comportamento definido explicitamente (como `private` ou `shared`), ou serem declaradas dentro do escopo da diretiva.
- **`firstprivate`**: cria cópias privadas das variáveis para cada thread, inicializando-as com o valor que tinham antes da região paralela (geralmente da thread mestre).
- **`lastprivate`**: a variável privada assume, ao final da execução, o valor correspondente à última iteração do loop (de acordo com a ordem sequencial).

Cláusula shared, private, default



```
#pragma omp parallel for shared (a ,n) default ( none )  
for (i = 0; i < n; i ++ ) {  
    a[i] += i;  
}
```

Cláusula private x firstprivate x lastprivate

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main ( int argc , char * argv []) {
    int i ,qtd_threads = 4, n =2*qtd_threads , a =1000;
    omp_set_num_threads (qtd_threads) ;
    #pragma omp parallel <CLAUSE>(a)
    {
        #pragma omp for
        for (i = 0; i < n; i ++) {
            a += +1;
        }
        printf (" Thread %d tem um valor de a = %d para \n", omp_get_thread_num () ,a );
    }
    printf (" Thread master tem um valor de a = %d para \n" ,a );
    return (0) ;
}
```

Medindo o tempo

```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) { /* omp_parallel .c */
    double inicio , fim ;
    inicio = omp_get_wtime () ;
    # pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        #pragma omp for
        for (int i = 0; i < 12; i++)
        {
            printf (" Alo da thread %d. fazendo o trabalho %d \n", tid, i );
        }
    }
    fim = omp_get_wtime () ;
    printf ("Tempo gasto na execução = %f segundos \n", fim - inicio);
    return 0;
}

// Compilar
// gcc -o my_omp_parallel_program omp_parallel.c - fopenmp
// Executar
// ./my_omp_parallel_program
```

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA - OPENMP

Tarefa 6:

Implemente em C a estimativa estocástica de π . Paralelize com

`#pragma omp parallel for`

e explique o resultado incorreto. Corrija a condição de corrida utilizando o

`#pragma omp critical`

e reestruturando com

`#pragma omp parallel`

seguido de

`#pragma omp for`

e aplicando as cláusulas `private`,

`firstprivate`, `lastprivate` e `shared`.

Teste diferentes combinações e explique como cada cláusula afeta o comportamento do programa. Comente também como a cláusula `default(none)` pode ajudar a tornar o escopo mais claro em programas complexos.

- Parallel e parallel for
- Escopo de variáveis
- Condição de corrida e regiões críticas

Referências

